

PROTOTYPE DEVELOPMENT PLAN

EVSE Prototype Development Phase Document

Smart AC Electric Vehicle Charging Station — STM32F401 + ESP32 Architecture

Prepared by:	Jaiyeola Emmanuel
Role:	Embedded Systems Engineer (Prototype Lead)
Prepared for:	Momentum
Date:	July 28, 2026
Document status:	Draft for review — Prototype Development Phase
Classification:	Internal — for review purposes

Table of Contents

1. Executive Summary

This document defines the development plan for a prototype AC Electric Vehicle Supply Equipment (EVSE) charging station, built around a two-microcontroller architecture: an STM32F401 acting as the real-time, safety-critical charging controller, and an ESP32 acting as the high-level connectivity, UI, and cloud-communication controller. The two microcontrollers communicate over UART using a custom framed protocol described in Section 7.

The core design principle of the prototype is a hard separation between safety authority and application logic: the STM32 independently validates every condition required to energize the charging contactor, and the ESP32 can only request charging actions — it can never command the contactor directly. This mirrors the architecture of real EVSE products while remaining transparent about what is prototype-grade versus production/certified-grade work.

This document is intended to serve three purposes: (1) a step-by-step engineering build guide, (2) a project status and planning reference for stakeholder review, and (3) technical documentation suitable for portfolio and future reference.

Note: Items marked *TBD*, *Assumption*, or *Requires Verification* throughout this document are open engineering decisions that will be resolved as the project progresses, and are flagged intentionally rather than guessed at.

2. Project Objectives

- Build a working single-phase AC EVSE prototype demonstrating the full charging lifecycle: connection detection, authentication, charging, telemetry, and session termination.
- Demonstrate a safe, layered embedded architecture separating real-time safety control from connectivity and application logic.
- Implement a robust UART protocol between the STM32 and ESP32 with framing, acknowledgment, and fail-safe behavior.
- Integrate OCPP (via microOCPP) for standards-aligned CSMS communication, and MQTT for engineering telemetry and diagnostics.
- Provide an LVGL-based touch UI for local interaction.
- Produce documentation, diagrams, and a repository structure suitable for engineering review and long-term maintenance.
- Clearly delineate prototype scope versus production/certification requirements — no compliance claims are made that have not been validated.

3. System Requirements

3.1 Functional Requirements

- Detect EV connection and disconnection via Control Pilot (CP) and Proximity Pilot (PP) signaling.
- Measure voltage, current, power, and cumulative energy.
- Authenticate a user or session before charging begins.

- Start and stop charging safely via contactor control.
- Display real-time charging status and session data on a local TFT UI.
- Report telemetry and transaction data to a backend and/or CSMS.
- Detect and respond to faults: overcurrent, overvoltage, undervoltage, communication loss, and emergency stop.

3.2 Non-Functional Requirements

- The STM32 must be able to reach and maintain a safe state independently of ESP32 availability.
- UART communication loss must be detected within a bounded time (TBD — e.g. three heartbeat intervals; Assumption).
- The system must be modular enough that OCPP, MQTT, and UI components can be developed and tested independently.
- Firmware should be structured for future portability toward a production-style architecture, without claiming production certification.

3.3 Out of Scope for This Prototype

- Formal IEC 61851 / IEC 61980 / SAE J1772 compliance testing (Requires Verification if pursued later).
- Three-phase charging.
- DC fast charging (this is an AC EVSE only).
- Formal safety certification — no ISO 26262-style hazard analysis is performed at prototype stage.

4. Proposed System Architecture

The prototype is organized into three logical domains: the vehicle interface, the EVSE itself (split between the STM32 safety domain and the ESP32 application domain), and the cloud/backend. Figure 1 shows the high-level block diagram.

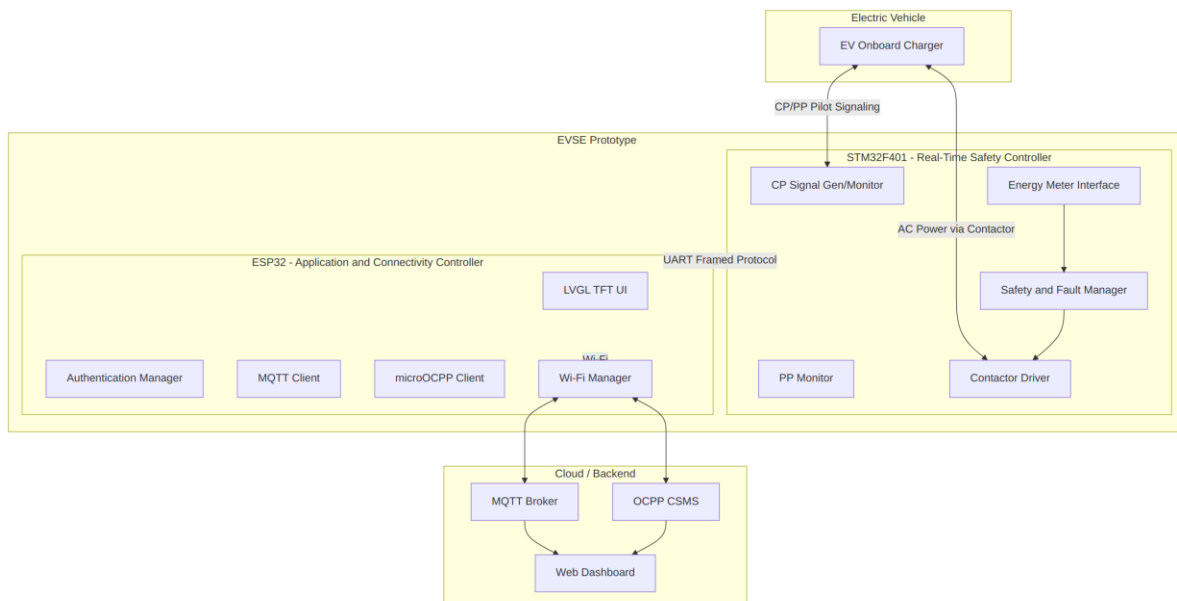


Figure 1 — High-level system block diagram.

4.1 Data Flow

Figure 2 illustrates how raw sensor data becomes validated telemetry, and how user/backend requests flow back toward the STM32 as non-authoritative requests only.

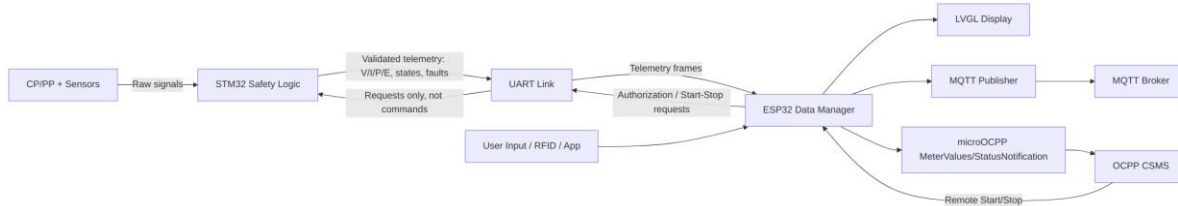


Figure 2 — Data flow diagram.

Design intent: every arrow into the STM32 from the ESP32 side is a request; STM32-internal logic is the only path that can assert control over the contactor.

5. STM32F401 Architecture

Role: real-time safety authority and charging controller.

5.1 Responsibilities

- CP signal generation and monitoring (PWM duty-cycle based state detection)
- PP monitoring (cable current-capacity detection)
- EV connection and disconnection detection
- Voltage, current, power, and energy measurement (via external energy meter)
- Contactor / relay control — sole authority
- Fault detection: overcurrent, overvoltage, undervoltage, CP/PP fault, communication timeout
- Emergency stop handling
- Hardware watchdog and fail-safe logic
- UART communication with ESP32 (telemetry out, requests in)

5.2 Firmware Architecture

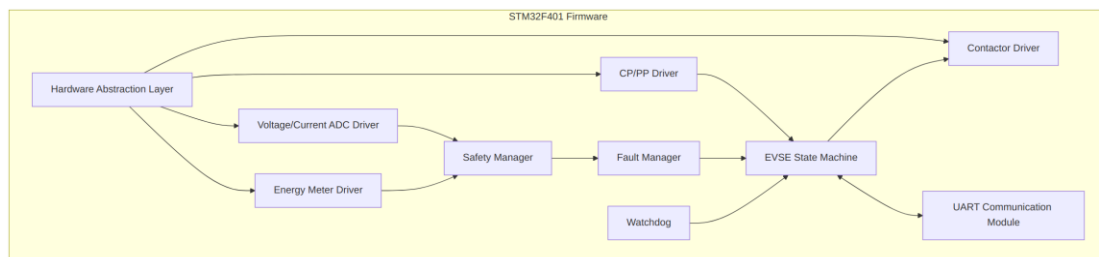


Figure 3 — STM32F401 firmware architecture.

The STM32 must independently verify EV connection, valid CP/PP state, absence of active faults, emergency-stop inactive status, voltage within limits, and current-protection availability before closing the contactor — regardless of any START_CHARGING_REQUEST received from the ESP32.

6. ESP32 Architecture

Role: high-level application, connectivity, and UI controller. This is explicitly not a safety-critical component.

6.1 Responsibilities

- LVGL TFT UI (status, session info, energy/power display)
- User authentication (local and/or backend-verified)
- Wi-Fi connectivity and reconnection handling
- MQTT client (telemetry and diagnostics)
- microOCPP client (CSMS transactions) — subject to Requires Verification against the specific microOCPP version's supported feature profiles
- UART communication with STM32 (sending requests, receiving telemetry)
- Forwarding data to backend and dashboard

6.2 Firmware Architecture

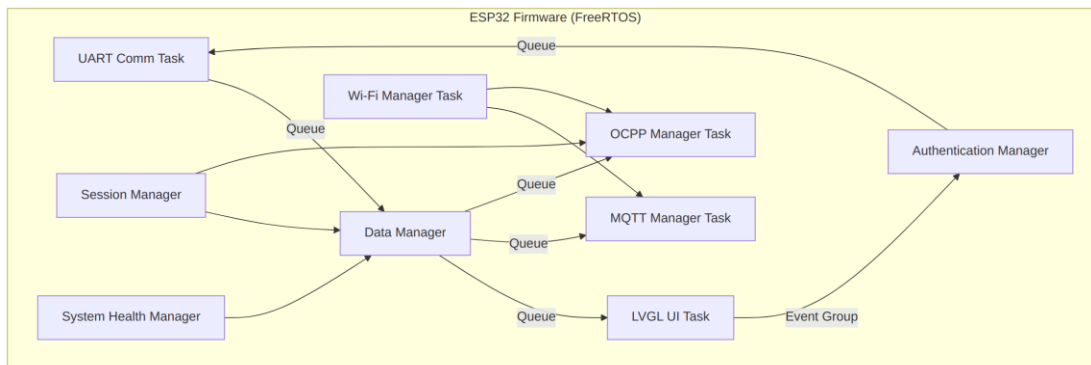


Figure 4 — ESP32 firmware architecture (FreeRTOS tasks).

6.3 FreeRTOS Task Design Notes

- UART Comm Task — highest relevant priority among non-safety tasks; owns the UART peripheral; never blocks on network I/O.
- LVGL UI Task — runs on its own task/timer; reads shared state via mutex-protected structures or a message queue; never blocks on Wi-Fi, MQTT, or OCPP calls.
- Wi-Fi / MQTT / OCPP Tasks — run independently and communicate results back via queues, not direct function calls into the UI task.
- Shared telemetry state is protected with a mutex, or passed via a single-producer queue from the UART task, to avoid race conditions with the display refresh.
- Communication primitives: queues for telemetry/command passing, event groups for state transitions (e.g. Wi-Fi connected, user authenticated), mutexes for shared data structures.

7. STM32 ↔ ESP32 Communication Architecture (UART Protocol)

7.1 Frame Structure

Field	Size	Description
Start-of-Frame (SOF)	1 byte	Fixed value, e.g. 0xAA
Message Type	1 byte	Command / Telemetry / ACK / NACK / Heartbeat
Command ID	1 byte	Identifies the specific message (see Section 7.2)
Sequence Number	1 byte	Increments per message, wraps at 255; used for ACK matching and duplicate detection
Payload Length	1 byte	Number of payload bytes (0–255)
Payload	N bytes	Message-specific data
CRC	2 bytes	CRC-16 over Message Type through Payload
End-of-Frame (EOF)	1 byte	Fixed value, e.g. 0x55

7.2 Message Table

Message	Direction	Description	Payload (draft)
EV_CONNECTED	STM32→ESP32	EV plug detected	none
EV_DISCONNECTED	STM32→ESP32	EV unplugged	none
CP_STATE	STM32→ESP32	Current CP state (A/B/C/D/E/F)	1 byte enum
PP_STATE	STM32→ESP32	Cable current capacity	1 byte enum
CHARGING_READY	STM32→ESP32	All safety checks passed	none
CONTACTOR_STATUS	STM32→ESP32	Open / closed / fault	1 byte enum
VOLTAGE_UPDATE	STM32→ESP32	RMS voltage	4-byte float
CURRENT_UPDATE	STM32→ESP32	RMS current	4-byte float
POWER_UPDATE	STM32→ESP32	Active power	4-byte float
ENERGY_UPDATE	STM32→ESP32	Cumulative session energy	4-byte float
FAULT_ACTIVE	STM32→ESP32	Fault code raised	1 byte fault code
FAULT_CLEARED	STM32→ESP32	Fault code cleared	1 byte fault code
CHARGING_STARTED	STM32→ESP32	Contactor closed, charging began	none
CHARGING_STOPPED	STM32→ESP32	Charging ended, reason code	1 byte reason
SESSION_STATUS	STM32→ESP32	Composite state snapshot	TBD struct
HEARTBEAT	Both	Liveness signal	none

Message	Direction	Description	Payload (draft)
START_CHARGING_REQUEST	ESP32→STM32	Requests charging start (not authoritative)	none
STOP_CHARGING_REQUEST	ESP32→STM32	Requests charging stop	1 byte reason
RESET_FAULT	ESP32→STM32	Requests fault reset (STM32 re-validates before clearing)	1 byte fault code
GET_STATUS	ESP32→STM32	Poll full status	none
SET_CURRENT_LIMIT	ESP32→STM32	Requests a charging current limit	4-byte float
AUTHORIZATION_GRANTED	ESP32→STM32	User/session authorized	session ID (TBD)
AUTHORIZATION_DENIED	ESP32→STM32	Authorization denied	none

Important: all ESP32→STM32 messages above are requests. Only the STM32's internal safety manager decides whether to act. AUTHORIZATION_GRANTED unlocks the STM32's ability to consider a start request — it does not itself start charging.

7.3 Reliability Mechanisms

- ACK/NACK: every Command-type message expects an ACK/NACK with a matching sequence number within a timeout window (TBD — e.g. 200 ms; Assumption).
- Retransmission: up to N retries (TBD — e.g. 3) before the sender treats the link as degraded.
- Heartbeat: both sides send HEARTBEAT at a fixed interval (TBD — e.g. 1 s; Assumption). Missing three consecutive heartbeats triggers link-loss handling.
- STM32 fail-safe on UART loss: STM32 continues independent safety monitoring; if charging is active it may continue under previously validated conditions but must reject further ESP32 requests, escalating to a connectivity-degraded fault if loss persists beyond a defined threshold (Assumption — exact policy needs an engineering decision).
- ESP32 behavior on UART loss: display a clear “STM32 not responding” state and disable start/stop controls.

7.4 UART Communication Sequence

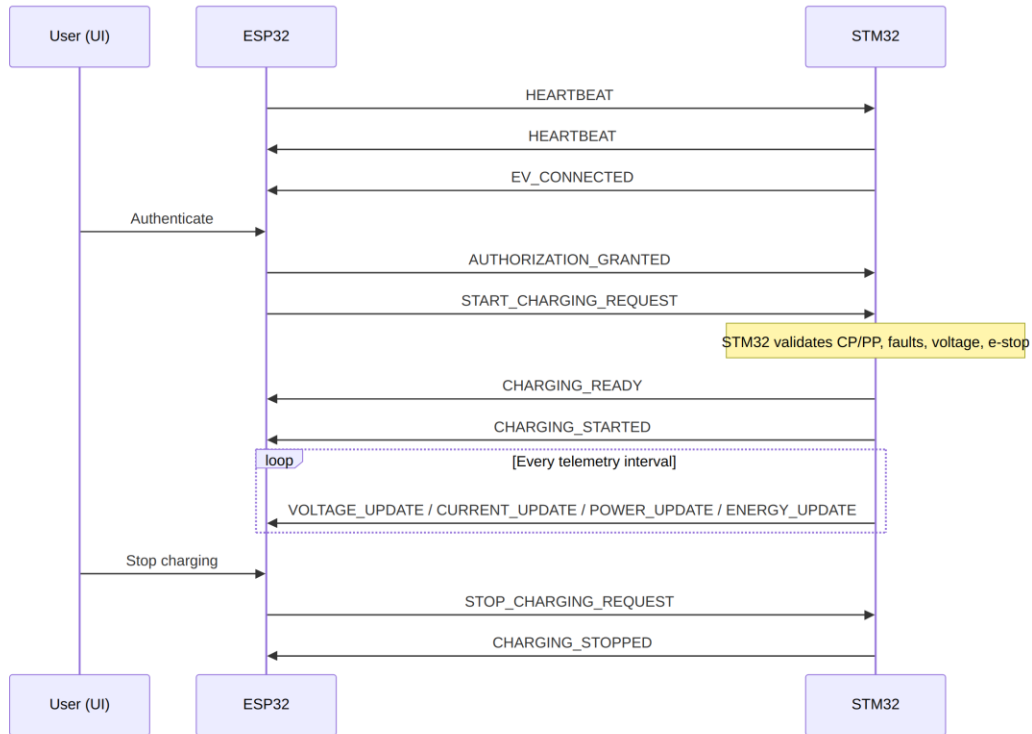


Figure 5 — Typical UART message sequence for a charging session.

8. EVSE Control and State Machine

8.1 Charging Sequence (Narrative)

- 1. EVSE powers on; STM32 initializes hardware and runs self-test.
- 2. ESP32 initializes, brings up LVGL, connects to Wi-Fi, initializes MQTT/OCPP.
- 3. STM32 begins CP/PP monitoring.
- 4. EV is plugged in; STM32 detects connection and notifies ESP32.
- 5. ESP32 displays “EV Connected”; user authenticates (local or backend).
- 6. ESP32 sends AUTHORIZATION_GRANTED, then START_CHARGING_REQUEST.
- 7. STM32 independently verifies all safety conditions.
- 8. STM32 transitions to CHARGING_READY, then closes the contactor and enters CHARGING.
- 9. STM32 streams telemetry; ESP32 updates the UI and publishes via MQTT/OCPP.
- 10. Charging ends (user stop, remote stop, or fault); contactor opens; session finalized.
- 11. ESP32 reports final session data to backend/CSMS; system returns to IDLE.

8.2 EVSE State Machine

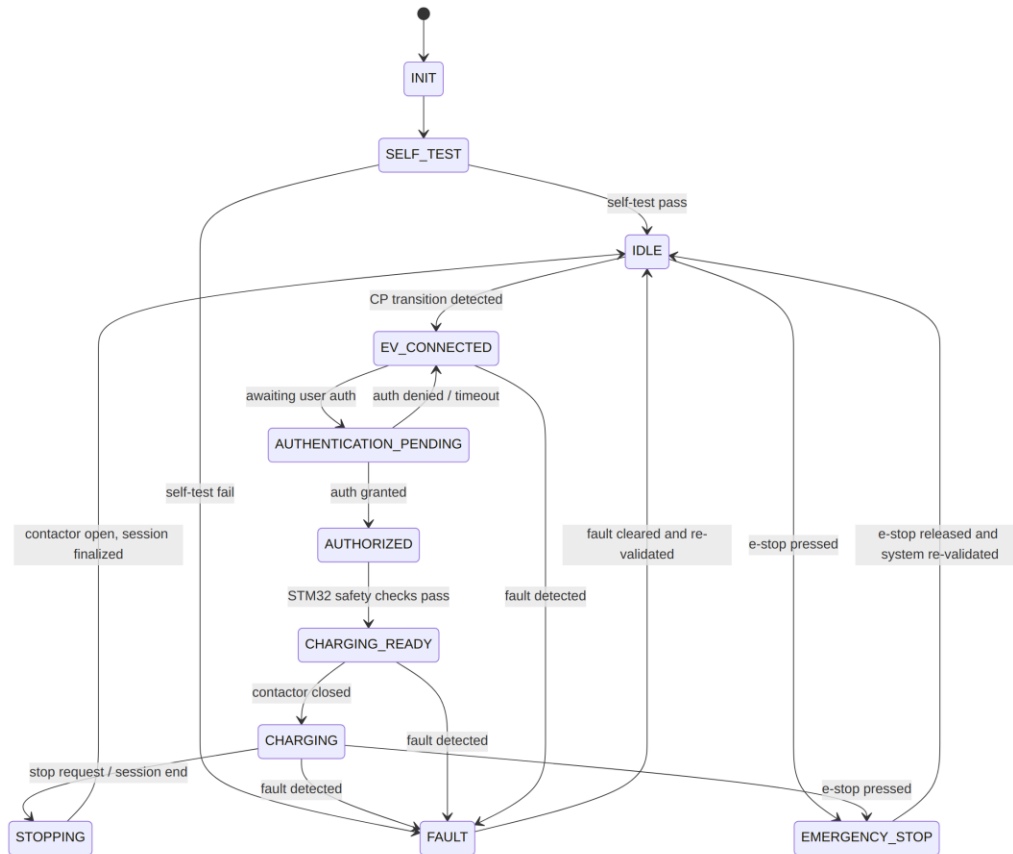


Figure 6 — EVSE state machine.

9. CP/PP and Charging end Control

- Control Pilot (CP): the STM32 generates the 1 kHz PWM pilot signal and reads the resulting analog voltage levels to determine EV state (A: not connected, B: connected/not ready, C: charging, D: charging with ventilation required, E/F: error). Duty cycle communicates available current to the EV. Exact ADC thresholds are Requires Verification against the reference circuit used.
- Proximity Pilot (PP): a resistor-coded signal read by the STM32 to determine cable current rating, used as an input to the current-limit calculation (minimum of PP-indicated capacity and configured supply capacity).
- The STM32 treats CP state C (or D) plus a valid PP plus no active fault as necessary but not sufficient — final go/no-go also requires voltage/current protection readiness and emergency-stop status.

10. Metering and Energy Measurement

- The prototype may use a single-phase energy meter module (e.g. a GY-series meter) for voltage, current, power, and energy readings — exact model TBD.
- The STM32 communicates with the meter over its native interface (UART / I2C / pulse output — TBD depending on the chosen module) and converts readings into the internal telemetry format sent to the ESP32.

- Overcurrent, overvoltage, and undervoltage thresholds must be defined per the target supply and hardware ratings (TBD) and enforced independently of the meter reading path where possible (e.g. a hardware comparator or a redundant ADC-based check) to avoid a single point of failure — Assumption, subject to hardware design review.

11. Safety and Fault Management

Safety philosophy: the STM32 is the sole safety authority. The ESP32 is explicitly non-safety-critical.

Condition	STM32 Behavior
Loss of ESP32 / UART timeout	Reject new start requests; continue independent monitoring; move toward safe/fault state if prolonged
Overcurrent	Immediate contactor open, latch fault
Overvoltage / Undervoltage	Immediate contactor open, latch fault
CP/PP invalid or fault state	Prevent or terminate charging
Emergency stop pressed	Immediate contactor open, enter EMERGENCY_STOP, requires manual/validated reset
Contactor feedback mismatch	Latch fault (suspected welded/stuck contactor) — detection feasibility Requires Verification against chosen contactor/relay hardware
Sensor/meter failure	Latch fault, disallow charging start
Watchdog timeout (STM32 hang)	Hardware watchdog forces reset; contactor defaults to open on reset (fail-safe wiring)

Prototype vs. production: this document does not claim compliance with IEC 61851, IEC 60364, UL 2594/2231, or similar standards. Any safety logic described here is a prototype-level implementation for demonstration and learning purposes; production certification would require formal hazard analysis, redundant sensing, and third-party testing.

12. LVGL User Interface

Screen	Information Displayed	User Actions	Data Required
Boot / Splash	Logo, firmware version	None	None
Idle	“Plug in vehicle”, Wi-Fi/OCPP status	None	Connection state, connectivity status
EV Connected	CP/PP state, “Ready to authenticate”	Tap to authenticate	CP_STATE, PP_STATE
Authentication	RFID/app prompt, status	Scan / tap / login	Auth result
Charging	Voltage, current, power, energy, duration, session/auth status	Stop charging	All telemetry fields, session ID
Session Summary	Total energy, duration, cost (if applicable)	Dismiss / new session	Final ENERGY_UPDATE, tariff data
Fault	Fault code/description, guidance	Acknowledge / reset (if	FAULT_ACTIVE payload

Screen	Information Displayed	User Actions	Data Required
		permitted)	
Settings / Diagnostics	Firmware versions, network info, current-limit config	Adjust settings	SESSION_STATUS, GET_STATUS response

13. Authentication

- Prototype authentication method: TBD — options include RFID tag, mobile app tap-to-start, or PIN entry on the touchscreen.
- Authentication result must be explicitly communicated to the STM32 as AUTHORIZATION_GRANTED / AUTHORIZATION_DENIED, which the STM32 stores as a precondition (not a bypass) for a subsequent START_CHARGING_REQUEST.
- If OCPP is used, Authorize.req / Authorize.conf (1.6J) or AuthorizeRequest / AuthorizeResponse (2.0.1) can validate against the CSMS — Requires Verification against the microOCPP feature profile in use.

14. MQTT Architecture

Role: engineering telemetry, diagnostics, and internal IoT monitoring — not the transactional record of charging sessions.

14.1 Suggested Topic Structure (TBD, illustrative)

```
evse/<device_id>/telemetry/voltage
evse/<device_id>/telemetry/current
evse/<device_id>/telemetry/power
evse/<device_id>/telemetry/energy
evse/<device_id>/status/state
evse/<device_id>/status/fault
evse/<device_id>/diagnostics/heap
evse/<device_id>/diagnostics/uart_link
```

- QoS level: TBD, likely QoS 1 for telemetry, QoS 0 acceptable for high-frequency diagnostics.
- MQTT should never be the sole path for transactional (billing/session) data — that responsibility belongs to OCPP.

15. OCPP Architecture

Role: authoritative EVSE-to-CSMS transactional communication. OCPP handles BootNotification/Heartbeat, Authorize, StartTransaction/TransactionEvent, StopTransaction/TransactionEvent, MeterValues, StatusNotification, and RemoteStartTransaction/RemoteStopTransaction.

15.1 OCPP 1.6J vs. OCPP 2.0.1 for This Prototype

Factor	OCPP 1.6J	OCPP 2.0.1
--------	-----------	------------

Factor	OCPP 1.6J	OCPP 2.0.1
microOCPP library maturity on ESP32	Generally the more established path	Feature completeness Requires Verification for the specific microOCPP version used
Message complexity	Simpler, smaller JSON payloads	Richer device model, more complex, higher RAM/flash footprint
Ecosystem / CSMS availability	Broad support among test CSMS platforms	Growing but less universal support in hobbyist/test CSMS tools
Practicality for this prototype	Recommended	Consider only if a specific CSMS target requires 2.0.1

Recommendation: OCPP 1.6J is the more practical choice for this ESP32 + microOCPP prototype, given lower resource requirements and broader current library/CSMS support. This should be re-confirmed against the exact microOCPP release in use (Requires Verification).

15.2 MQTT vs. OCPP Role Separation

	OCPP	MQTT
Purpose	Transactional / billing-relevant	Engineering / diagnostic
Data	MeterValues, StartTransaction, StopTransaction, Authorize	Raw telemetry, heap usage, link health
Consumer	CSMS	Internal dashboard, developer tools

Advantage of separation: avoids overloading OCPP with high-frequency diagnostic noise and keeps transactional records clean and standards-aligned. Disadvantage: some data duplication (e.g. power/energy sent via both channels) and two connections to maintain and monitor.

16. Backend and Web Dashboard

- A backend/dashboard component is optional for the initial prototype phases but recommended for stakeholder demonstration value.
- Suggested responsibilities: receive MQTT telemetry, receive/relay OCPP transaction data (via a CSMS or a lightweight custom OCPP server), and display live and historical session data.
- Specific backend stack: TBD (e.g. a Node.js/Python service plus a time-series or relational database and a simple web frontend).

17. Firmware Architecture (Summary)

Firmware architecture is covered in detail in Sections 5 and 6. The key cross-cutting principle is that no blocking network or UI operation may interfere with safety-relevant timing — this is enforced structurally by keeping UART communication, safety logic, and network/UI logic in separate tasks/cores with queue-based handoffs.

18. Hardware Development Plan (Phased)

Twelve development phases take the project from basic microcontroller bring-up to full system integration. Each phase lists its objective, required hardware and software, key tasks, expected output, test procedure, acceptance criteria, and known risks.

Phase 1 — STM32F401 Basic Firmware and GPIO Validation

Objective	Bring up the STM32 toolchain and verify GPIO, clocks, and basic peripherals.
Hardware	STM32F401 dev board, ST-Link, LEDs/buttons for bring-up.
Software	STM32CubeIDE/CubeMX, HAL drivers.
Tasks	Clock configuration, GPIO toggle test, UART echo test.
Expected Output	Blinking LED; UART loopback confirmed.
Test Procedure	Toggle GPIO at a known frequency; verify with oscilloscope/logic analyzer.
Acceptance Criteria	Stable clock, correct GPIO timing, working UART echo.
Risks	Clock misconfiguration; incorrect pin mapping.

Phase 2 — Voltage and Current Sensing

Objective	Validate the ADC-based or meter-based voltage/current acquisition path.
Hardware	Voltage divider/isolation circuit, current sensor (CT or shunt — TBD).
Software	ADC driver, calibration routine.
Tasks	Implement ADC sampling, apply calibration constants, validate against a known load.
Expected Output	Accurate voltage/current readings within tolerance (TBD target).
Test Procedure	Compare readings against a calibrated multimeter/clamp meter.
Acceptance Criteria	Error within the defined tolerance across the test range.
Risks	Noise; incorrect scaling; isolation/safety hazards when working with mains voltage — appropriate isolation and caution required.

Phase 3 — Energy Meter Integration

Objective	Integrate the chosen energy meter module.
Hardware	Energy meter module (model TBD).
Software	Meter communication driver (protocol per module datasheet — Requires Verification).
Tasks	Implement read routine, parse meter data, cross-check against Phase 2 sensing.
Expected Output	Consistent V/I/P/E readings from the meter module.
Test Procedure	Run a known load; compare meter output to reference.
Acceptance Criteria	Readings consistent and stable over time.
Risks	Communication protocol quirks; incorrect scaling factors.

Phase 4 — CP and PP Implementation

Objective	Implement CP PWM generation/monitoring and PP resistor-code reading.
Hardware	CP/PP signal conditioning circuit (TBD reference design).
Software	Timer-based PWM, ADC-based CP voltage-level decode, PP threshold decode.
Tasks	Generate the CP signal, decode EV state, decode PP cable rating.
Expected Output	Correct state detection across A/B/C transitions.
Test Procedure	Simulate CP/PP states with test resistors or an EV simulator.
Acceptance Criteria	Correct state decoded across all test conditions.
Risks	Incorrect threshold calibration; noise on the pilot line.

Phase 5 — Contactor/Relay Control

Objective	Implement safe contactor drive circuitry and control logic.
Hardware	Contactor/relay and driver circuit, feedback sensing if available.
Software	Contactor driver module with interlocks.
Tasks	Implement open/close logic gated by the safety manager.
Expected Output	Contactor closes only when all preconditions are met.
Test Procedure	Attempt closure under valid and invalid conditions; verify correct behavior.
Acceptance Criteria	Contactor never closes without full precondition validation.
Risks	Relay welding; incorrect flyback protection; mains safety hazards.

Phase 6 — Fault Detection and Safety Logic

Objective	Implement the fault manager and safety state transitions.
Hardware	As above.
Software	Fault manager, safety manager, state machine integration.
Tasks	Implement overcurrent/overvoltage/undervoltage detection and e-stop input handling.
Expected Output	Correct fault latching and safe-state transitions.
Test Procedure	Inject simulated fault conditions.
Acceptance Criteria	All fault conditions result in contactor open within the defined response time (TBD).
Risks	Missed edge cases; race conditions in the state machine.

Phase 7 — UART Communication with ESP32

Objective	Implement the framed UART protocol (Section 7).
Hardware	STM32 and ESP32 connected via UART.
Software	Framing, CRC, ACK/NACK, heartbeat, and retransmission logic on both sides.
Tasks	Implement and test the protocol stack.
Expected Output	Reliable bidirectional messaging with correct error handling.

Test Procedure	Fault injection (disconnect UART, corrupt frames); verify recovery behavior.
Acceptance Criteria	No message loss under normal conditions; correct fail-safe behavior under fault injection.
Risks	Timing mismatches; buffer overflows; CRC edge cases.

Phase 8 — ESP32 LVGL UI

Objective	Implement UI screens per Section 12.
Hardware	TFT display and touch controller.
Software	LVGL, display driver, touch driver.
Tasks	Build all screens; wire to the Data Manager.
Expected Output	Functional UI reflecting live STM32 telemetry.
Test Procedure	Manual UI walkthrough across all states.
Acceptance Criteria	All screens render correctly and update in real time.
Risks	LVGL task blocking; memory constraints.

Phase 9 — Wi-Fi and MQTT

Objective	Implement Wi-Fi connectivity and MQTT publishing.
Hardware	ESP32 onboard Wi-Fi.
Software	Wi-Fi manager, MQTT client library.
Tasks	Connect/reconnect logic, topic structure, publish telemetry.
Expected Output	Reliable MQTT telemetry stream.
Test Procedure	Disconnect/reconnect Wi-Fi and verify recovery; subscribe with an MQTT client to confirm data.
Acceptance Criteria	Automatic reconnection; correct topic/payload structure.
Risks	Network instability; credential management.

Phase 10 — microOCPP Integration

Objective	Integrate the OCPP 1.6J transactional flow.
Hardware	N/A (software phase).
Software	microOCPP library.
Tasks	BootNotification, Authorize, StartTransaction/StopTransaction, MeterValues, StatusNotification.
Expected Output	Successful transaction lifecycle against a test CSMS.
Test Procedure	Run against an open-source test CSMS (TBD which one).
Acceptance Criteria	Full transaction lifecycle completes without protocol errors.
Risks	Library feature gaps — Requires Verification against microOCPP's supported feature profiles.

Phase 11 — Backend/Web Dashboard Integration

Objective	Connect MQTT/OCPP data to a dashboard.
Hardware	N/A.
Software	Backend stack (TBD).
Tasks	Ingest MQTT/OCPP data; build a simple live dashboard.
Expected Output	Live session/telemetry visualization.
Test Procedure	Run an end-to-end session; verify the dashboard reflects it.
Acceptance Criteria	Dashboard shows accurate real-time and historical data.
Risks	Data model mismatches between MQTT and OCPP sources.

Phase 12 — Full System Integration

Objective	Combine all subsystems into one working prototype.
Hardware	Complete assembled prototype.
Software	All modules integrated.
Tasks	End-to-end integration testing and bug fixing.
Expected Output	Fully functioning EVSE prototype.
Test Procedure	Full end-to-end test per Section 21.
Acceptance Criteria	Complete charging cycle succeeds reliably across repeated trials.
Risks	Integration-level bugs not visible in isolated module testing.

19. Software Development Plan

The software plan mirrors the hardware phases above. STM32 firmware modules (Phases 1–6) and ESP32 firmware modules (Phases 8–9) can largely be developed in parallel once the UART protocol (Phase 7) is specified, since both sides need to agree on the frame format early.

20. Integration Plan

- 1. Bench-test the STM32 and ESP32 independently (Phases 1–6 and 8–9 respectively).
- 2. Connect via UART and validate the protocol (Phase 7) using mocked data on each side before relying on real sensor/CP data.
- 3. Integrate real CP/PP and metering data into the STM32 side.
- 4. Integrate OCPP/MQTT into the ESP32 side.
- 5. Perform full hardware integration (contactor, wiring, enclosure).
- 6. Run full end-to-end and fault-injection testing (Section 21).

21. Testing and Validation Plan

21.1 Test Categories

- Unit testing (individual firmware modules)
- Hardware testing (sensors, relay, CP/PP, meter)
- Communication testing (UART STM32↔ESP32)
- Fault injection (UART loss, ESP32/STM32 reboot, Wi-Fi loss, MQTT/OCPP loss, sensor disconnect, overcurrent, e-stop)
- Integration testing (full system)
- End-to-end testing (connection → authentication → charging → telemetry → termination)
- Safety testing (unsafe conditions always prevent or terminate charging)

21.2 Representative Test Cases

ID	Description	Initial Condition	Procedure	Expected Result
TC-01	CP state decode	EV simulator set to state B	Connect simulator, read CP state	STM32 reports state B within one sample cycle
TC-02	Overcurrent trip	Charging active	Inject current above threshold	Contactor opens, fault latched, trip time within limit (TBD)
TC-03	UART link loss	Charging active	Physically disconnect UART	STM32 continues safe monitoring; ESP32 shows link-loss UI; no unsafe contactor behavior
TC-04	Unauthorized start attempt	EV connected, no auth	Send START_CHARGING_REQUEST without AUTHORIZATION_GRANTED	STM32 rejects request; contactor remains open
TC-05	Emergency stop	Charging active	Press e-stop	Immediate contactor open within defined response time (TBD)
TC-06	ESP32 reboot mid-charge	Charging active	Reset ESP32	STM32 maintains or safely terminates charging per defined fail-safe policy
TC-07	Wi-Fi loss	Session active, telemetry publishing	Disable Wi-Fi	MQTT/OCPP disconnect gracefully; reconnect on restore; no crash
TC-08	Full end-to-end session	System idle	Plug in EV → authenticate → charge → unplug	Full lifecycle completes; session data matches expected energy/duration within tolerance

ID	Description	Initial Condition	Procedure	Expected Result
TC-09	Contactor feedback mismatch	Contactor commanded open	Simulate stuck-closed feedback	Fault latched; charging disallowed until cleared
TC-10	CRC-corrupted UART frame	Normal operation	Inject corrupted frame	Receiver discards frame and awaits retransmission; no invalid command executed

22. Development Roadmap

Stage	Description	Depends On	Parallel With
1. Requirements	Finalize functional/non-functional requirements	—	—
2. System Architecture	Define block diagram, responsibilities	Stage 1	—
3. Hardware Design	Schematic/PCB for sensing, CP/PP, contactor	Stage 2	Stage 5 (early ESP32 work)
4. STM32 Firmware	Phases 1–6	Stage 3	Stage 5
5. ESP32 Firmware	Phases 8–9 (UI, Wi-Fi/MQTT)	Stage 2	Stage 4
6. UART Protocol	Phase 7	Stages 4 & 5 basic bring-up	—
7. EVSE Control	State machine integration on STM32	Stage 4	—
8. UI	Full LVGL screens	Stage 5	Stage 9
9. MQTT	Telemetry publishing	Stage 5	Stage 8
10. OCPP	microOCPP integration	Stage 9	—
11. Backend	Dashboard	Stages 9 & 10	—
12. Integration	Combine STM32 + ESP32 + hardware	Stages 6, 7, 8	—
13. Testing	Full test plan (Section 21)	Stage 12	—
14. Documentation	Finalize this document, diagrams, repo	Ongoing, finalized after Stage 13	—

23. Risk Assessment

Risk	Impact	Likelihood	Mitigation
Mains-voltage hardware hazards during bring-up	High	Medium	Use isolation, low-voltage bench testing first, follow proper safety practice
microOCPP feature gaps vs. requirements	Medium	Medium	Verify supported feature profiles early (Phase 10)
UART protocol edge cases	Medium	Medium	Dedicated fault-injection testing (Phase 7,

Risk	Impact	Likelihood	Mitigation
(corruption, timing)			TC-10)
Energy meter module selection delay	Low–Medium	Medium	Treat as TBD; use ADC-based interim measurement (Phase 2)
Scope creep (adding DC/three-phase)	Medium	Low	Explicitly out of scope per Section 3.3

24. GitHub / Project Repository Structure

```

evse-prototype/
├── firmware/
│   ├── stm32/
│   │   ├── Core/
│   │   ├── Drivers/
│   │   ├── Safety/
│   │   └── Communication/
│   └── esp32/
│       ├── main/
│       ├── ui/
│       ├── network/
│       └── ocpp/
├── hardware/
│   ├── schematics/
│   └── pcb/
├── protocol/
│   └── uart-protocol-spec.md
├── backend/
├── web-dashboard/
├── docs/
│   ├── EVSE_Prototype_Development_Phase_Document.docx
│   └── diagrams/
├── tests/
│   ├── unit/
│   ├── integration/
│   └── fault-injection/
├── simulation/
└── research/

```

25. Engineering Deliverables

- System block diagram and data-flow diagram
- Hardware schematic and PCB design files
- STM32 firmware source
- ESP32 firmware source
- UART protocol specification document
- EVSE state machine diagram
- LVGL UI implementation
- MQTT topic/architecture documentation

- OCPP integration report
- Backend/web dashboard (optional for MVP)
- Test report (results of Section 21 test cases)
- Fault analysis summary
- This technical documentation set

26. Future Improvements

- Three-phase charging support
- Formal safety certification pathway
- OCPP 2.0.1 migration if a target CSMS requires it
- Load-balancing across multiple EVSE units
- Dynamic tariff/billing integration
- Mobile app for remote monitoring/control

27. Conclusion

This document defines a layered, safety-first architecture for a prototype AC EVSE, separating real-time safety control (STM32F401) from connectivity and application logic (ESP32). It is intended as both a practical build guide and a stakeholder-facing planning reference. Items marked TBD, Assumption, or Requires Verification should be resolved and updated as the project progresses; this document should be treated as a living artifact tracked in version control alongside the codebase.

Prepared for internal review. Feedback and open questions are welcome before hardware procurement and PCB layout begin.